

Recurrent Neural Networks

Bùi Huỳnh Tây - 24521589

Đoàn Hồng Bảo - 24520005

Nguyễn Thành Sơn - 24521532

Nguyễn Quốc Phú - 24520025

Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM

Ngày 4 tháng 5 năm 2026

- 1 Recurrent Neural Networks
- 2 RNNs as Language Models
- 3 RNNs for other NLP tasks
- 4 Stacked and Bidirectional RNN architectures

13.1 Recurrent Neural Networks (RNN)

- **Mục đích:** Xử lý dữ liệu tuần tự (văn bản, time series).
- **Đặc trưng:** Kiến trúc có **chu trình** (cycle).
- **Bộ nhớ:** Lưu thông tin quá khứ qua trạng thái ẩn (hidden state).

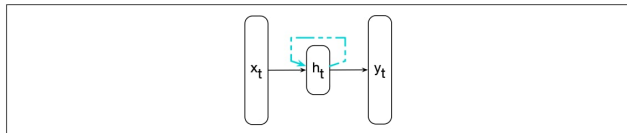


Figure 13.1 Simple recurrent neural network after [Elman \(1990\)](#). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

Hình: Kiến trúc RNN với vòng lặp

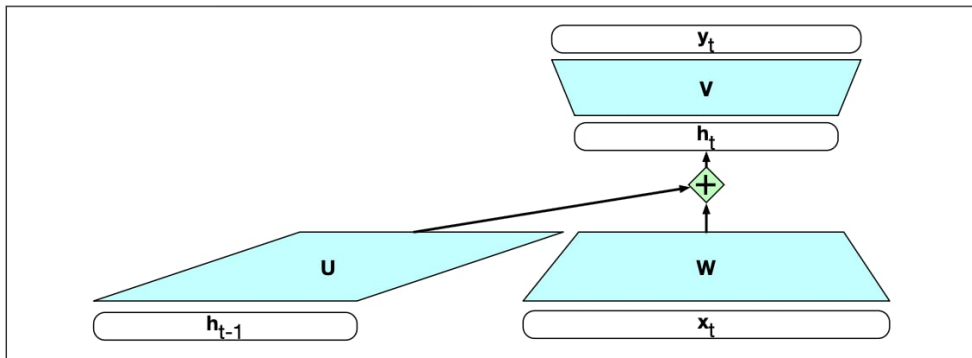


Figure 13.2 Simple recurrent neural network illustrated as a feedforward network. The hidden layer $\mathbf{h}_{t-1}$ from the prior time step is multiplied by weight matrix $\mathbf{U}$ and then added to the feedforward component from the current time step.

Hình: Trãi phảng một bước RNN

Forward Inference (Truyền xuôi)

Phương trình

$$h_t = \tanh(Uh_{t-1} + Wx_t)$$

$$y_t = \text{softmax}(Vh_t)$$

- U : Trọng số hồi quy.
- W : Trọng số đầu vào hiện tại.
- V : Trọng số đầu ra.
- **Cốt lõi:** Trọng số **dùng chung** (shared) ở mọi bước.

function FORWARDRNN($\mathbf{x}$, $network$) **returns** output sequence $\mathbf{y}$

```
 $\mathbf{h}_0 \leftarrow 0$ 
for  $i \leftarrow 1$  to LENGTH( $\mathbf{x}$ ) do
     $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$ 
     $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$ 
return  $\mathbf{y}$ 
```

Figure 13.3 Forward inference in a simple recurrent network. The matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

Bài tập

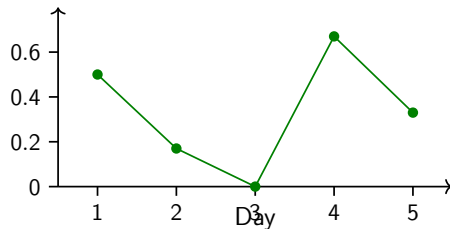
Công thức:

$$h_t = \tanh(U \cdot h_{t-1} + W \cdot x_t)$$

$$\hat{y}_t = V \cdot h_t$$

Thông số:

- $U = 0.9, W = 1.5, V = 0.8$
- $h_0 = 0$



Ngày (t)	1	2	3	4	5
Input (x_t)	0.50	0.17	0.00	0.67	0.33
Target (y_t)	0.17	0.00	0.67	0.33	0.17
Ẩn (h_t)	?	?	?	?	?
Dự đoán ($\hat{y}_t$)	?	?	?	?	?

Đáp án Forward Inference

- Cập nhật: $h_t = \tanh(0.9 \cdot h_{t-1} + 1.5 \cdot x_t)$
- Dự đoán: $\hat{y}_t = 0.8 \cdot h_t$

Ngày (t)	1	2	3	4	5
Input (x_t)	0.50	0.17	0.00	0.67	0.33
Target (y_t)	0.17	0.00	0.67	0.33	0.17
Ẩn (h_t)	0.635	0.678	0.544	0.904	0.864
Dự đoán ($\hat{y}_t$)	0.508	0.542	0.435	0.723	0.691

Backpropagation Through Time: Mục tiêu

1. Hàm Loss toàn chuỗi:

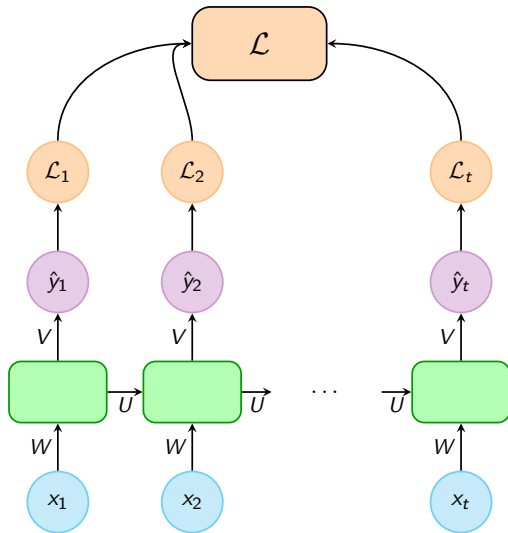
$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$$

2. Đạo hàm toàn phần theo U :

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial U}$$

3. Chain Rule tại bước t :

$$\frac{\partial \mathcal{L}_t}{\partial U} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \frac{d\mathbf{h}_t}{dU}$$



Backpropagation Through Time: Multivariable Chain Rule

Quy tắc chuỗi đa biến

$$f(x) = f(p(x), q(x)) \implies \frac{\partial f}{\partial x} = \frac{\partial f}{\partial p} \frac{\partial p}{\partial x} + \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Áp dụng vào trạng thái ẩn $\mathbf{h}_t$:

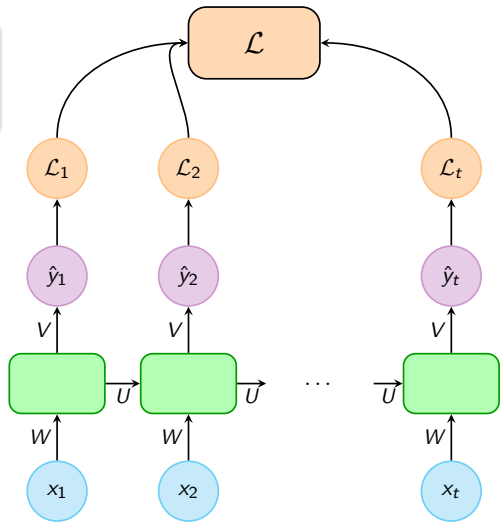
$$\mathbf{h}_t = \sigma_h(W\mathbf{x}_t + U\mathbf{h}_{t-1}) = \mathbf{h}_t(W, \mathbf{x}_t, p(U), q(U))$$

$\mathbf{h}_t$ phụ thuộc vào U theo hai nhánh:

- **Trực tiếp:** $p(U) = U$
- **Gián tiếp:** $q(U) = \mathbf{h}_{t-1}$

Khai triển đạo hàm toàn phần:

$$\frac{\partial \mathbf{h}_t}{\partial U} = \underbrace{\frac{\partial \mathbf{h}_t}{\partial U}}_{\text{Trực tiếp}} \cdot \underbrace{\frac{\partial U}{\partial U}}_{=1} + \underbrace{\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial U}}_{\text{Gián tiếp}}$$



Backpropagation Through Time

1. Công thức truy hồi:

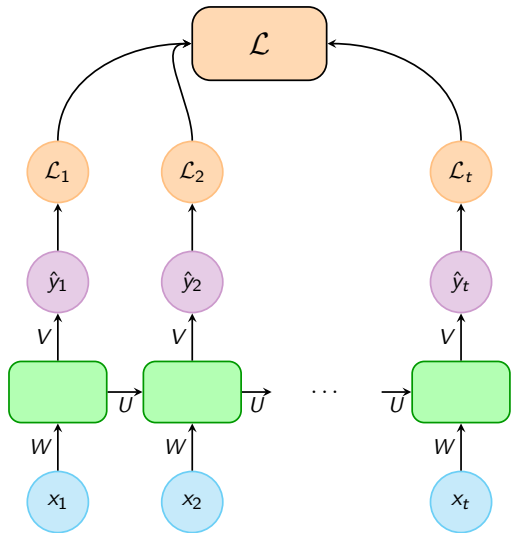
$$\frac{d\mathbf{h}_t}{dU} = \frac{\partial \mathbf{h}_t}{\partial U} + \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{d\mathbf{h}_{t-1}}{dU}$$

2. Khai triển qua các bước $k \leq t$:

$$\frac{d\mathbf{h}_t}{dU} = \sum_{k=1}^t \left(\prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{h}_k}{\partial U}$$

3. Lỗi lan truyền từ bước k :

$$\left(\frac{\partial \mathbf{h}_t}{\partial U} \right)_k = \left(\prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \mathbf{h}_k}{\partial U}$$



Backpropagation Through Time: Phân rã Jacobian

1. Đặt biến trung gian $\mathbf{z}_i$:

$$\mathbf{z}_i = W\mathbf{x}_i + U\mathbf{h}_{i-1} + \mathbf{b}_h \implies \mathbf{h}_i = \sigma_h(\mathbf{z}_i)$$

2. Tách mắt xích Jacobian:

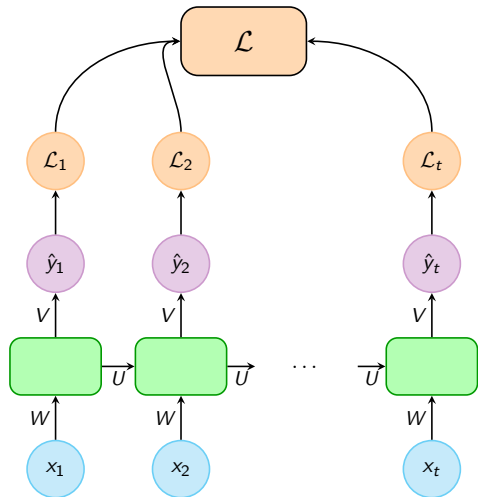
$$\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \frac{\partial \mathbf{h}_i}{\partial \mathbf{z}_i} \cdot \frac{\partial \mathbf{z}_i}{\partial \mathbf{h}_{i-1}}$$

3. Giải thích ma trận đường chéo $\frac{\partial \mathbf{h}_i}{\partial \mathbf{z}_i}$:

Hàm σ_h được áp dụng độc lập lên từng phần tử ($h_{i,k} = \sigma_h(z_{i,k})$).
Suy ra đạo hàm chéo bằng 0:

$$\frac{\partial h_{i,k}}{\partial z_{i,j}} = \begin{cases} \sigma'_h(z_{i,k}) & \text{nếu } k = j \\ 0 & \text{nếu } k \neq j \end{cases}$$
$$\implies \frac{\partial \mathbf{h}_i}{\partial \mathbf{z}_i} = \begin{bmatrix} \sigma'_h(z_{i,1}) & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma'_h(z_{i,n}) \end{bmatrix} = \text{diag}(\sigma'_h(\mathbf{z}_i))$$

4. Đạo hàm theo $\mathbf{h}_{i-1}$: $\frac{\partial \mathbf{z}_i}{\partial \mathbf{h}_{i-1}} = U$



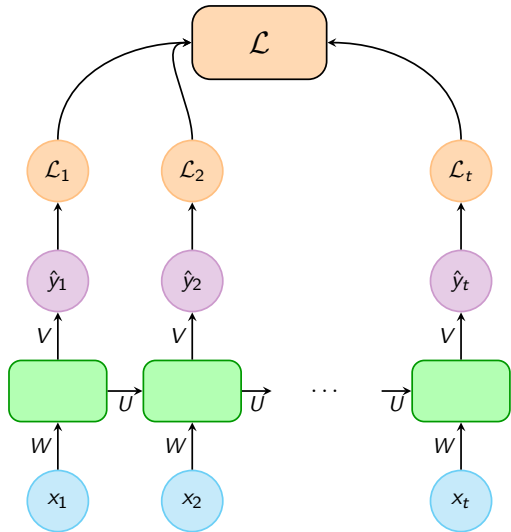
Backpropagation Through Time: Ký hiệu Ω

1. Ma trận chuyển trạng thái Ω_i :

$$\frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \underbrace{\text{diag}(\sigma'_h(\mathbf{z}_i))}_{\frac{\partial \mathbf{h}_i}{\partial \mathbf{z}_i}} \cdot \underbrace{U}_{\frac{\partial \mathbf{z}_i}{\partial \mathbf{h}_{i-1}}} = \Omega_i$$

2. Chuỗi lan truyền lỗi:

$$\begin{aligned} \left(\frac{\partial \mathbf{h}_t}{\partial U} \right)_k &= \left(\prod_{j=k+1}^t \Omega_j \right) \frac{\partial \mathbf{h}_k}{\partial U} \\ &= \Omega_t \cdot \Omega_{t-1} \cdots \Omega_{k+1} \frac{\partial \mathbf{h}_k}{\partial U} \end{aligned}$$



Đánh giá Recurrent Neural Networks (RNN)

Ưu điểm (Advantages)

- **Độ dài đầu vào linh hoạt:** Khác với mạng nơ-ron truyền thống (Feedforward), RNN có thể xử lý chuỗi dữ liệu có độ dài bất kỳ.
- **Tối ưu tham số:** Tham số mô hình (U, W, V) được *chia sẻ* qua tất cả các bước thời gian, giúp kích thước mô hình không bị phình to theo độ dài chuỗi.
- **Bộ nhớ ngữ cảnh:** Trạng thái ẩn h_t mang theo thông tin lịch sử, giúp mô hình hiểu được ngữ cảnh của chuỗi.

Điểm yếu (Disadvantages)

- **Tính toán tuần tự:** Bước t bắt buộc phải đợi kết quả từ bước $t - 1$. Điều này gây khó khăn cho việc song song hóa, khiến tốc độ huấn luyện chậm.
- **Trí nhớ ngắn hạn:** Gặp khó khăn trong việc liên kết các thông tin ở khoảng cách quá xa (Long-term dependencies).
- **Vanishing/Exploding Gradient:** Khi dùng BPTT trên chuỗi dài, việc nhân liên tiếp các ma trận Jacobian Ω dễ làm đạo hàm bị triệt tiêu (tiến về 0) hoặc bùng nổ (tiến tới ∞).

- 1 Recurrent Neural Networks
- 2 RNNs as Language Models**
- 3 RNNs for other NLP tasks
- 4 Stacked and Bidirectional RNN architectures

13.2 RNNs as Language Models

- **Language Models (LM):** Mô hình ngôn ngữ dự đoán từ tiếp theo trong chuỗi dựa trên ngữ cảnh các từ trước đó: $P(w_{t+1}|w_1, \dots, w_t)$.
- Xác suất của toàn bộ chuỗi được phân rã bằng quy tắc chuỗi (chain rule):

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i|w_{<i}) \quad (1)$$

- **Khắc phục hạn chế của các mô hình cũ:**
 - *N-gram LM*: Ngữ cảnh bị giới hạn cứng ở $n - 1$ từ trước đó.
 - *Feedforward LM (Chương 6)*: Bị giới hạn bởi kích thước cửa sổ đầu vào cố định.
 - *RNN LM*: Xử lý từng từ một, trạng thái ẩn h_{t-1} có thể đại diện cho toàn bộ thông tin từ tận đầu chuỗi mà không bị giới hạn độ dài.

13.2.1 Forward Inference in an RNN Language Model

Các bước tính toán tại thời điểm t

$$e_t = Ex_t \quad (2)$$

$$h_t = g(Uh_{t-1} + We_t) \quad (3)$$

$$\hat{y}_t = \text{softmax}(Vh_t) \quad (4)$$

- $X = [x_1, \dots, x_N]$: Chuỗi đầu vào, mỗi từ là một vector one-hot.
- E : Ma trận nhúng từ (word embedding matrix).
- $\hat{y}_t$: Phân phối xác suất dự đoán qua toàn bộ từ vựng V .

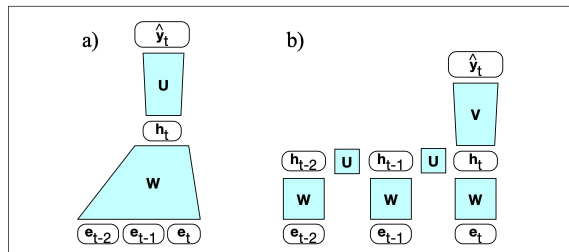


Figure 13.5 Simplified sketch of two LM architectures moving through a text, showing a schematic context of three tokens: (a) a feedforward neural language model which has a fixed context input to the weight matrix $\mathbf{W}$, (b) an RNN language model, in which the hidden state h_{t-1} summarizes the prior context.

Hình: FFN LM (trái) vs RNN LM (phải)

Chi tiết về Chiều (Dimensions) và Xác suất

- Giả sử chiều nhúng (embedding dimension) và chiều lớp ẩn (hidden dimension) bằng nhau: $d_e = d_h = d$.
- Kích thước các ma trận và vector:**
 - $E \in \mathbb{R}^{d \times |V|}$, vector one-hot $x_t \in \mathbb{R}^{|V| \times 1} \Rightarrow e_t \in \mathbb{R}^{d \times 1}$
 - $W, U \in \mathbb{R}^{d \times d} \Rightarrow h_t \in \mathbb{R}^{d \times 1}$
 - $V \in \mathbb{R}^{|V| \times d} \Rightarrow Vh_t \in \mathbb{R}^{|V| \times 1}$
- Xác suất để từ k trong từ vựng là từ tiếp theo chính là thành phần thứ k của vector $\hat{y}_t$:

$$P(w_{t+1} = k | w_1, \dots, w_t) = \hat{y}_t[k] \quad (5)$$

- Xác suất của toàn bộ chuỗi (w_i là từ thực tế tại vị trí i):

$$P(w_{1:n}) = \prod_{i=1}^n \hat{y}_i[w_i] \quad (6)$$

13.2.2 Training an RNN Language Model

- **Self-supervision:** Dùng chính trình tự của chuỗi văn bản làm nhãn giám sát.
- Dùng **Cross-Entropy Loss** để đo sự khác biệt giữa dự đoán $\hat{y}_t$ và phân phối chuẩn y_t :

$$L_{CE}(\hat{y}_t, y_t) = - \sum_{w \in V} y_t[w] \log \hat{y}_t[w] \quad (7)$$

- Do y_t là vector one-hot (chỉ có giá trị 1 tại từ thực tế w_{t+1}), công thức được rút gọn:

$$L_{CE}(\hat{y}_t, y_t) = - \log \hat{y}_t[w_{t+1}] \quad (8)$$

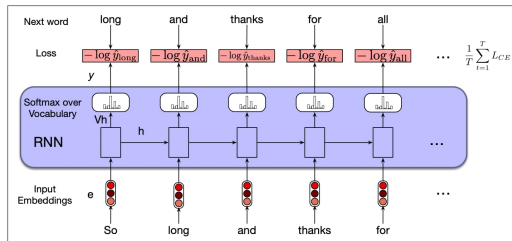


Figure 13.6 Training RNNs as language models.

Hình: Huấn luyện RNN LM

Teacher Forcing & 13.2.3 Weight Tying

- **Teacher Forcing:** Khi huấn luyện, để tính từ w_{t+2} , mô hình luôn được nạp từ thực tế (gold word) w_{t+1} làm đầu vào, thay vì dùng dự đoán $\hat{y}_t$ có thể bị sai của bước trước.
- **Weight Tying (Liên kết trọng số):**
 - Cả ma trận nhúng $E \in \mathbb{R}^{d \times |V|}$ và ma trận đầu ra $V \in \mathbb{R}^{|V| \times d}$ đều có hình dạng chuyển vị của nhau và cùng làm nhiệm vụ biểu diễn từ.
 - Thay vì dùng ma trận V riêng biệt, ta tái sử dụng E^T ở lớp Softmax. Điều này giúp giảm thiểu số lượng tham số và tăng hiệu suất ngôn ngữ.

Cập nhật phương trình với Weight Tying

$$e_t = Ex_t \quad (9)$$

$$h_t = g(Uh_{t-1} + We_t) \quad (10)$$

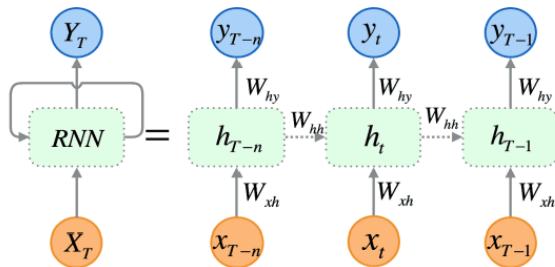
$$\hat{y}_t = \text{softmax}(E^T h_t) \quad (11)$$

- 1 Recurrent Neural Networks
- 2 RNNs as Language Models
- 3 RNNs for other NLP tasks**
- 4 Stacked and Bidirectional RNN architectures

13.3 RNNs for other NLP tasks

Trong NLP, mô hình RNN thường được áp dụng trong 3 loại công việc như sau:

- **Sequence classification:** Điển hình là sentiment analysis và topic classification
- **Sequence labeling:** Gán nhãn cho chuỗi
- **Text generation tasks:** Dùng cơ chế sinh văn bản tự hồi quy



Hình: mô hình RNN

13.3.1 Sequence Labeling

- **Định nghĩa:** Gán một nhãn từ tập cố định cho *mỗi* phần tử của chuỗi.
- **Ví dụ điển hình:** Part-of-speech tagging.
- **Kiến trúc:**
 - Đầu vào: Word embeddings (đã pre-trained).
 - Đầu ra: Xác suất nhãn qua lớp softmax tại mỗi bước.

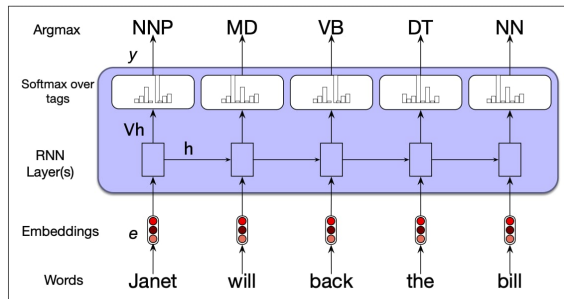


Figure 13.7 Part-of-speech tagging as sequence labeling with a simple RNN. The goal of part-of-speech (POS) tagging is to assign a grammatical label to each word in a sentence, drawn from a predefined set of tags. (The tags for this sentence include NNP (proper noun), MD (modal verb) and others; we'll give a complete description of the task of part-of-speech tagging in Chapter 17.) Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

Hình: POS Tagging as sequence labeling

13.3.1 Sequence Labeling: Inference & Training

- **Mô hình RNN:** Một mạng recurrent được trải phẳng với lớp đầu vào, lớp ẩn, lớp đầu ra tại mỗi bước. Dùng chung trọng số U, V, W .
- **Inference:** Duyệt qua chuỗi đầu vào $\rightarrow$ Chọn nhãn có xác suất cao nhất (Argmax) tại mỗi bước.
- **Training:** Sử dụng hàm mất mát **Cross-Entropy** do tính chất phân phối xác suất từ lớp softmax.

13.3.1 Tại sao MSE gây triệt tiêu đạo hàm?

1. Thành phần lớp đầu ra:

- Hàm kích hoạt Sigmoid: $\hat{y} = \sigma(z)$
- Đạo hàm: $\sigma'(z) = \hat{y}(1 - \hat{y})$

2. Hàm mất mát MSE:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

4. Hệ quả:

- Khi dự đoán sai rất nặng ($y = 1, \hat{y} \rightarrow 0$):

$$\frac{\partial L}{\partial z} \approx (0 - 1) \cdot \mathbf{0} = \mathbf{0}$$

⇒ **Mạng nơ-ron học vì tín hiệu lỗi bị triệt tiêu.**

3. Đạo hàm lan truyền ngược:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

$$\frac{\partial L}{\partial z} = (\hat{y} - y) \cdot \sigma'(z)$$

$$\frac{\partial L}{\partial z} = (\hat{y} - y) \cdot \underbrace{\hat{y}(1 - \hat{y})}_{\rightarrow 0 \text{ khi bão hòa}}$$

13.3.1 Cross-Entropy?

1. Hàm mất mát Cross-Entropy:

$$L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$$

2. Đạo hàm của CE theo dự đoán:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

3. Đạo hàm lan truyền ngược:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y})$$

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

13.3.2 Sequence Classification

- **Mục tiêu:** Phân loại *toàn bộ* chuỗi thay vì từng token (Text classification).
- **Ứng dụng:** Phân tích cảm xúc, Lọc thư rác, Phân loại chủ đề.
- **Cơ chế:**
 - Bỏ qua đầu ra ở các bước trung gian.
 - Dùng trạng thái ẩn cuối cùng (h_n) làm **biểu diễn nén** cho toàn chuỗi.

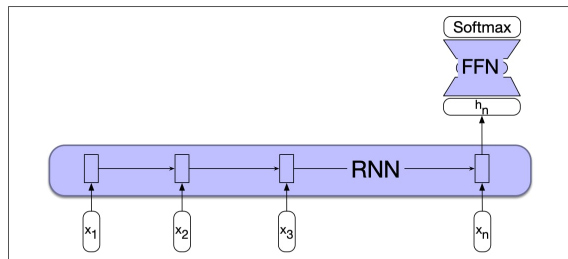


Figure 13.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

Hình: Sequence classification với RNN

13.3.2 Sequence Classification

- **Kiến trúc:**

- Truyền chuỗi đầu vào qua RNN từng bước.
- Đưa h_n qua mạng truyền thẳng (FFN) và dùng hàm softmax để chọn lớp.

- **Huấn luyện:**

- Loss chỉ tính ở đầu ra cuối, các từ trung gian không có giá trị loss.
- Lan truyền ngược qua FFN và xuyên suốt các bước của RNN.

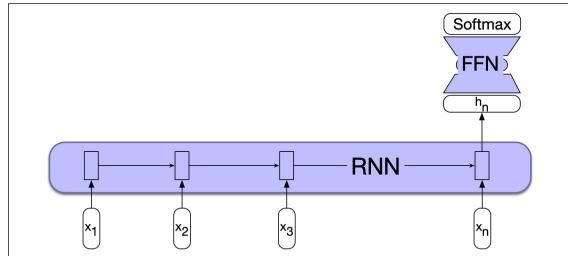


Figure 13.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

Hình: Sequence classification với RNN

13.3.2 Tại sao Lan truyền ngược lại khó?

Đạo hàm theo thời gian:

$$\frac{\partial L_n}{\partial h_1} = \frac{\partial L_n}{\partial h_n} \cdot \frac{\partial h_n}{\partial h_{n-1}} \cdot \frac{\partial h_{n-1}}{\partial h_{n-2}} \cdots \frac{\partial h_2}{\partial h_1}$$

Vấn đề Triệt tiêu đạo hàm:

- Mỗi mắt xích $\frac{\partial h_t}{\partial h_{t-1}}$ chứa ma trận trọng số W và đạo hàm kích hoạt σ' .
- Nếu các giá trị này < 1 , việc **nhân liên tiếp** qua n bước sẽ khiến:

$$\frac{\partial L_n}{\partial h_1} \rightarrow 0 \quad (\text{khi } n \text{ lớn})$$

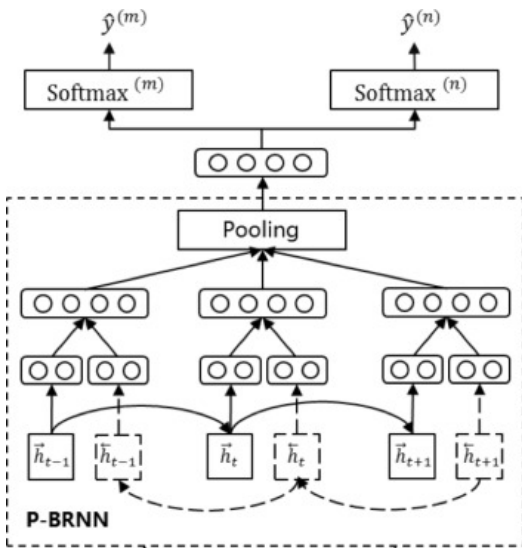
4. Hệ quả:

⇒ Mô hình bị "mất trí nhớ ngắn hạn", chỉ học được các từ gần cuối.

13.3.2 Sequence Classification

- **Suy giảm thông tin:** Thông tin của các từ ở đầu chuỗi rất dễ bị phai nhạt hoặc mất hẳn khi truyền qua nhiều bước đến cuối chuỗi.
- **Lan truyền ngược khó khăn:** Ngữ cảnh dài khiến tín hiệu lỗi phải đi một quãng đường quá xa từ h_n ngược về h_1 , gây khó khăn cho việc cập nhật trọng số và huấn luyện mạng hiệu quả.

13.3.2 Sequence Classification



Giải pháp:

- 1. **Element-wise Mean Pooling:** Lấy trung bình cộng theo từng phần tử:

$$h_{mean} = \frac{1}{n} \sum_{i=1}^n h_i$$

- 2. **Element-wise Max Pooling:** Lấy giá trị lớn nhất (max) tại mỗi chiều k từ tất cả n vector trạng thái ẩn.

13.3.2 Sequence Classification

- **Phương pháp dùng h_n (Trạng thái ẩn cuối):**

- **Ưu điểm:** Cấu trúc đơn giản, thuận tự nhiên, hiệu quả với các câu hoặc chuỗi ngắn.
- **Nhược điểm:** Bị nghẽn cổ chai, dễ quên thông tin đoạn đầu, khó khăn khi lan truyền ngược với văn bản dài.

- **Phương pháp dùng Pooling (h_{mean} hoặc h_{max}):**

- **Ưu điểm:** Thu thập được thông tin toàn cục từ mọi vị trí. Rút ngắn quãng đường lan truyền ngược, giải quyết rất tốt bài toán văn bản dài.
- **Nhược điểm:** Các phép toán Mean/Max coi các trạng thái như một tập hợp, làm **mất đi một phần thông tin về thứ tự** của từ ngữ trong câu.

13.3.3 Generation with RNN

- **Nguồn gốc:** Phát triển từ kỹ thuật lấy mẫu (sampling) của mô hình ngôn ngữ n-gram truyền thống.
- **Autoregressive Generation:** Sinh từ ngữ tăng dần ằng cách liên tục lấy mẫu từ tiếp theo dựa trên các từ đã chọn trước đó.

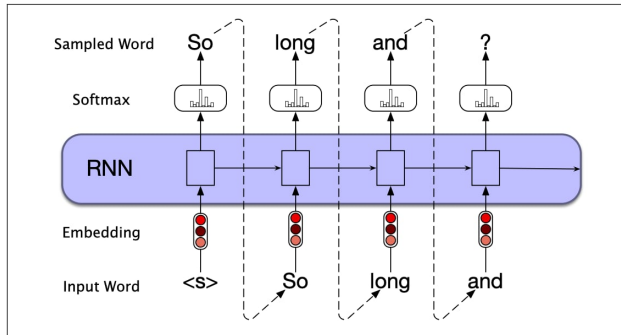


Figure 13.9 Autoregressive generation with an RNN-based neural language model.

Hình: Autoregressive generation với RNN

13.3.3 Generation

- Ứng dụng: **Dịch máy, Tóm tắt văn bản, Trả lời câu hỏi.**
- **Môi ngữ cảnh:**
 - *Dịch thuật:* Ngữ cảnh môi chính là toàn bộ câu ở ngôn ngữ gốc.
 - *Tóm tắt:* Ngữ cảnh môi là toàn bộ văn bản dài cần tóm tắt.
- Nhờ đó, các từ được sinh ra phía sau sẽ bị **ràng buộc chặt chẽ** vào đoạn ngữ cảnh môi này.

13.3.4 Bài tập

Đề bài: Một mạng RNN được dùng để phân tích câu: "*Phim - Rất - Hay*".

Cho các thông số mạng (đã được đơn giản hóa) như sau:

Trọng số mạng:

- W (ẩn $\rightarrow$ ẩn): $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
- U (vào $\rightarrow$ ẩn): $\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$
- V (ẩn $\rightarrow$ ra): $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

Đầu vào & Kích hoạt:

- x_1 (Phim) = $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$, x_2 (Rất) = $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- x_3 (Hay) = $\begin{bmatrix} 0 \\ 3 \end{bmatrix}$
- $h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- $f(z) = \text{ReLU}(z) = \max(0, z)$

Yêu cầu: Tính **Max Pooling** (h_{max}) cho toàn câu.

1. Tính h_1 :

$$z_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix}$$
$$\Rightarrow h_1 = \text{ReLU} \left(\begin{bmatrix} 2 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

2. Tính h_2 :

$$z_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$
$$\Rightarrow h_2 = \text{ReLU} \left(\begin{bmatrix} 0 \\ 2 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

3. Tính h_3 :

$$z_3 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} + \begin{bmatrix} -3 \\ 3 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$$
$$\Rightarrow h_3 = \text{ReLU} \left(\begin{bmatrix} -1 \\ 3 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

4. Tính Max Pooling:

$$\text{So sánh } h_1, h_2, h_3: \Rightarrow h_{\max} = \begin{bmatrix} \max(2, 0, 0) \\ \max(0, 2, 3) \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$\text{Đầu ra toàn câu: } y = V \cdot h_{\max} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$$

- 1 Recurrent Neural Networks
- 2 RNNs as Language Models
- 3 RNNs for other NLP tasks
- 4 Stacked and Bidirectional RNN architectures

Simple RNN

- RNN tích hợp yếu tố tuần tự thông qua **trạng thái ẩn** h_t , đóng vai trò như bộ nhớ lưu trữ ngữ cảnh quá khứ.

$$h_t = g(Uh_{t-1} + Wx_t)$$

- **Training RNNs:** Trải phẳng RNN $\rightarrow$ FFN và dùng Lan truyền ngược through time.
- **Ứng dụng:**
 - Language Model.
 - Sequence Classification / Labeling.
 - Text Generation.

Vấn đề đặt ra

Dù rất linh hoạt, RNN cơ bản chỉ đọc từ **trái sang phải** và **chỉ có một lớp ẩn**. Liệu điều này có đủ để hiểu được sự phức tạp và đa chiều của ngôn ngữ tự nhiên?

Dùng simple RNN giải bài toán nhận diện thực thể

Xét từ "**Teddy**":

- Câu 1: **Teddy** bears are on sale!.
- Câu 2: **Teddy** Roosevelt was a president.

Dùng simple RNN giải bài toán nhận diện thực thể

Xét bài toán nhận diện thực thể với từ **"Teddy"**:

- Câu 1: **Teddy** bears are on sale! → *"Teddy" là gấu bông (Danh từ chung).*
- Câu 2: **Teddy** Roosevelt was a president. → *"Teddy" là tên người (Danh từ riêng).*

Vấn đề: Khi mạng đọc đến từ **"Teddy"**, nó không có ngữ cảnh phía trước để ra quyết định chính xác. Nếu mô hình có thể truy cập toàn bộ chuỗi và nhìn sang các từ bên phải, việc dự đoán sẽ tốt hơn rất nhiều.

13.4.1 Stacked RNNs

- **Kiến trúc:** Xếp chồng nhiều mạng RNN. Đầu ra của lớp dưới làm đầu vào cho lớp trên.
- **Mục đích:** Giúp mạng học được các biểu diễn ở nhiều mức độ trừu tượng khác nhau qua từng lớp.
- Thường cho hiệu suất cao hơn mạng một lớp, nhưng chi phí huấn luyện tăng nhanh chóng.

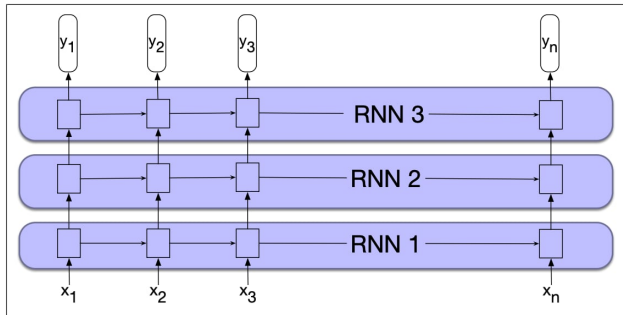


Figure 13.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

Hình: Stacked recurrent networks

Bài toán phân tích cảm xúc

Xét một câu đánh giá phim:

*"Bộ phim này rất **hấp dẫn** ở đoạn đầu, **nhưng** kết thúc lại quá **nhàm chán**."*

Câu hỏi: Làm sao mạng nơ-ron có thể hiểu được cảm xúc tổng thể của một câu có sự đảo chiều ngữ nghĩa như thế này?

Bài toán phân tích cảm xúc

Xét câu: *"Bộ phim này rất hấp dẫn ở đoạn đầu, nhưng kết thúc lại quá nhàm chán."*

Cách Stacked RNNs trích xuất mức độ trừu tượng qua từng lớp:

- **RNN 1 (Lớp thấp nhất - Từ vựng):** Nhận diện đặc trưng cơ bản cục bộ. ("hấp dẫn" → Tích cực; "nhàm chán" → Tiêu cực).
- **RNN 2 (Lớp giữa - Cú pháp):** Nắm bắt cấu trúc câu và từ nối. (VD: Nhận diện từ "nhưng" là một điểm đảo chiều / phủ định mệnh đề trước đó).
- **RNN 3 (Lớp cao nhất - Ngữ nghĩa toàn cục):** Tổng hợp thông tin từ lớp dưới → Nhấn mạnh vào vế sau chữ "nhưng" → Đưa ra kết luận cuối cùng: Cảm xúc tổng thể là **Tiêu cực**.

13.4.2 Bidirectional RNNs

- **Vấn đề:** RNN tiêu chuẩn chỉ dùng ngữ cảnh bên trái (quá khứ). Nhiều bài toán cần cả ngữ cảnh bên phải (tương lai).
- **Giải pháp:** Chạy hai mạng RNN độc lập:
 - Từ trái qua phải (Forward).
 - Từ phải qua trái (Backward).
- **Kết quả:** Concatenate hai biểu diễn lại để nắm bắt trọn vẹn ngữ cảnh hai chiều.

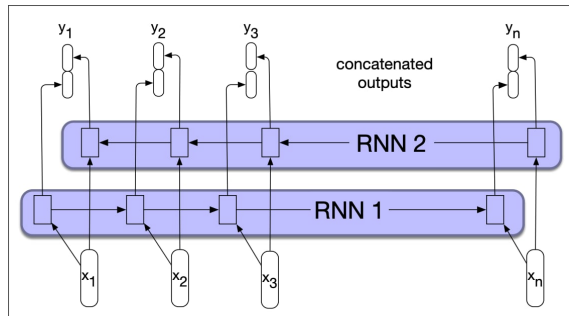


Figure 13.11 A bidirectional RNN. Separate models are trained in the forward and backward directions, with the output of each model at each time point concatenated to represent the bidirectional state at that time point.

Hình: A bidirectional RNN

13.4.2 Bidirectional RNNs: Equations

Các phương trình của Bi-RNN

- **Forward RNN:** Trạng thái ẩn h_t^f đại diện cho ngữ cảnh từ đầu chuỗi đến hiện tại t :

$$h_t^f = \text{RNN}_{\text{forward}}(x_1, \dots, x_t) \quad (13.16)$$

- **Backward RNN:** Trạng thái ẩn h_t^b đại diện cho ngữ cảnh từ cuối chuỗi n ngược về t :

$$h_t^b = \text{RNN}_{\text{backward}}(x_t, \dots, x_n) \quad (13.17)$$

- **Kết hợp:** Biểu diễn tại mỗi bước là phép nối của hai vector để nắm bắt ngữ cảnh hai chiều:

$$h_t = [h_t^f; h_t^b] = h_t^f \oplus h_t^b \quad (13.18)$$

Bidirectional RNNs for Sequence Classification

- **Nhược điểm RNN 1 chiều:** Trạng thái cuối h_n mang nhiều thông tin cuối câu hơn đầu câu.
- **Bi-RNN cho phân loại:** Kết hợp trạng thái ẩn cuối cùng của cả hai chiều truyền (forward + backward).
- Vector kết hợp này đại diện cho toàn bộ chuỗi và được truyền vào bộ phân loại (FFN).

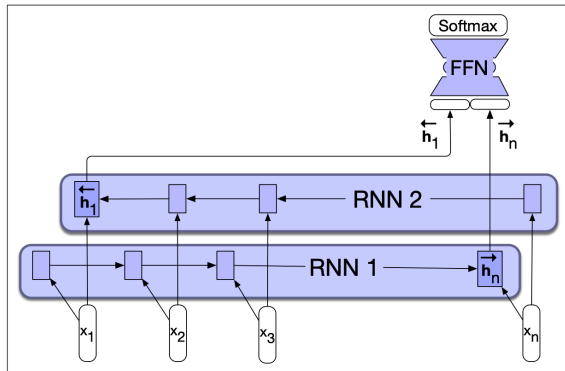


Figure 13.12 A bidirectional RNN for sequence classification. The final hidden units from the forward and backward passes are combined to represent the entire sequence. This combined representation serves as input to the subsequent classifier.

Hình: Bi-RNN cho Sequence Classification

Bài tập: Tính toán Bi-RNN

Đề bài: Tính z ?

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, x_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

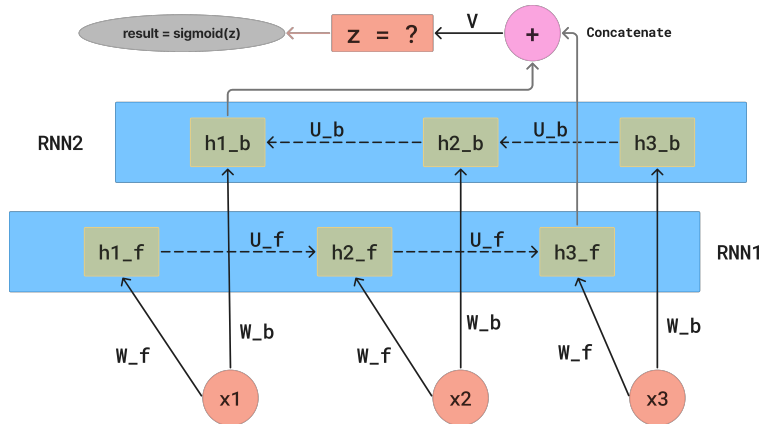
$$W_f = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, U_f = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$W_b = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, U_b = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$v = \begin{bmatrix} 1 & -1 & 0.5 & 0.5 \end{bmatrix}$$

$$h_{0_f} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, h_{4_b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$G() = \text{ReLU}()$$



1. Forward Pass ($\rightarrow$)

$$h_{t_f} = \text{ReLU}(U_f \cdot h_{t-1_f} + W_f \cdot x_t)$$

- $h_{1_f} = \text{ReLU} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
- $h_{2_f} = \text{ReLU} \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- $h_{3_f} = \text{ReLU} \left(\begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

3. Kết hợp & Dự đoán

- **Nối vector:** $H = [h_{3_f}; h_{1_b}] = [2, 2, 4, 4]^T$
- **Tính z:** $z = v \cdot H = \begin{bmatrix} 1 & -1 & 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 2 & 2 & 4 & 4 \end{bmatrix}^T = 2 - 2 + 2 + 2 = 4$

2. Backward Pass ($\leftarrow$)

$$h_{t_b} = \text{ReLU}(U_b \cdot h_{t+1_b} + W_b \cdot x_t)$$

- $h_{3_b} = \text{ReLU} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
- $h_{2_b} = \text{ReLU} \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 2 \end{bmatrix} \right) = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$
- $h_{1_b} = \text{ReLU} \left(\begin{bmatrix} 2 \\ 4 \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$